GROUP ASSIGNMENT 2 · BIG DATA · REGION 12: BENELUX (EUR)

The third city is where you find out whether you built a pipeline or a prototype

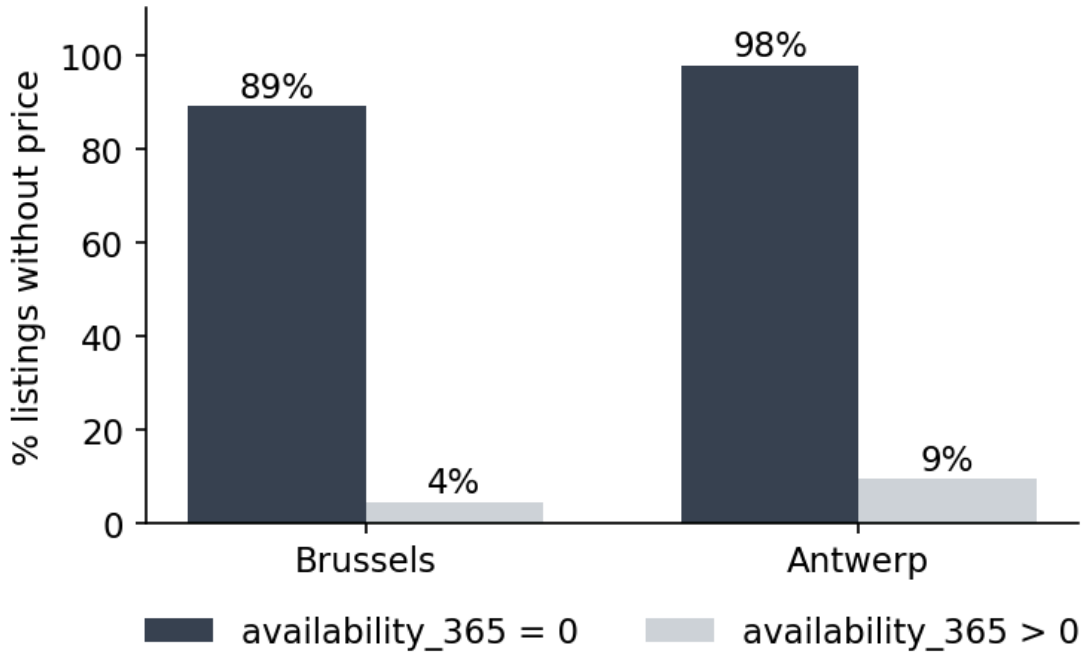
A Databricks lakehouse and price model for Brussels (A) and Antwerp (B), then run unchanged on the new client Amsterdam (C)

Team: *names*

The task, and what we built

- **Client cities A and B**: Brussels (snapshot 2026-09-20, 4,850 listings) and Antwerp (2026-09-23, 2,463)
- **New client C**: Amsterdam (2026-09-16, 10,612). Not downloaded until the A/B pipeline and model were frozen (git tag `week1-final`)
- Why Amsterdam: the largest and most expensive Benelux market, another country, another neighbourhood system. The hardest realistic transfer
- **Lakehouse**: bronze, silver and gold Delta tables in Unity Catalog, on Free Edition serverless (€0)
- **One pipeline, city as the only parameter**: a Databricks job for one city plus an orchestrator with for-each
- **Price model**: gradient boosting on log(price), against the required baseline
- **AI/BI dashboard** (3 pages) and a **Genie space** over the gold tables
- **City C report**: first run, every fix classified, the gap explained

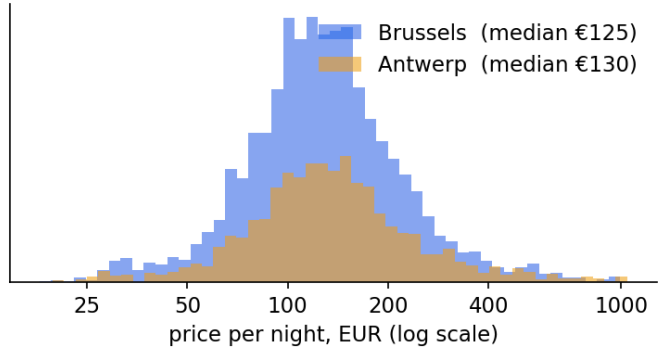
What the data told us: price is missing for a reason, and four columns are the price



Price is missing **not at random**: it is missing for listings with no bookable night (no quote). We exclude those rows from training (they are kept in silver for supply metrics) instead of inventing a price.

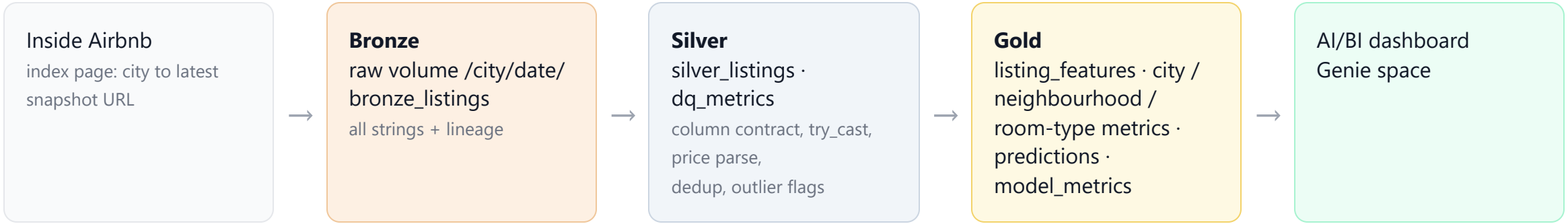
column	relation to price	verdict
price_quote_price_per_night	= price (99.96 %/100 %)	leak
price_quote_total_price	= price x nights	leak
price_quote_raw	JSON containing the price	leak
estimated_revenue_l365d	= price x occupancy (100 %)	leak
availability_*, occupancy	consequence of price	excluded
neighbourhood	target-encoded inside CV only	used

Also: 15 columns are 100 % empty in this snapshot; the long-stay (min. 28 nights) price is half (monthly discount).



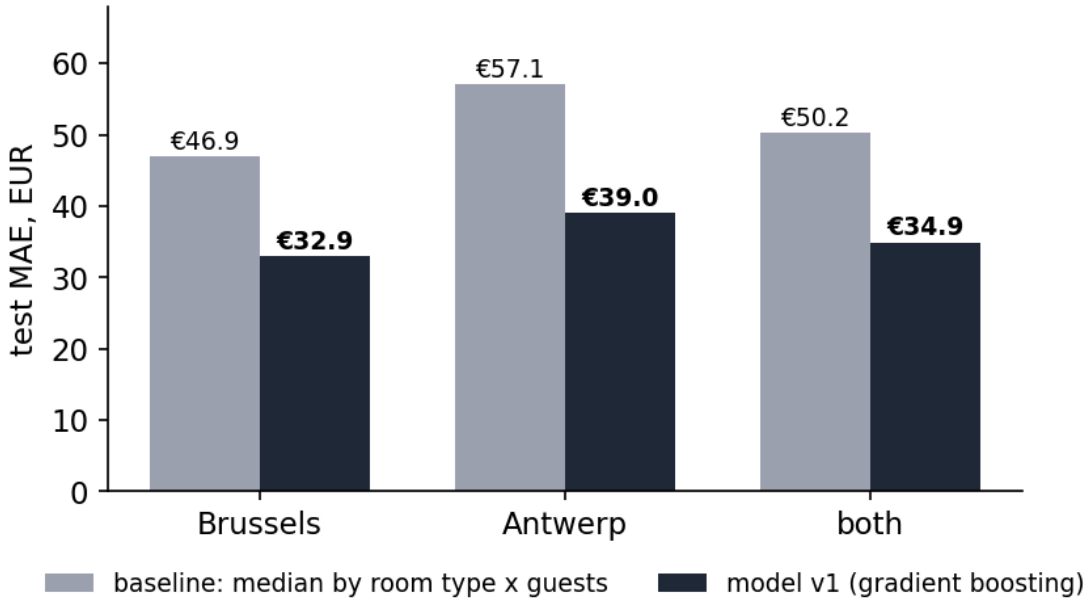
Prices are log-normal, so the model is trained on log(price); A and B have almost the same price level.

Architecture: medallion lakehouse, one parameterised pipeline



- **Job airbnb_city_etl**(city): bronze → silver → gold. **Job airbnb_lakehouse**(cities): for-each city ETL → train (optional) → for-each score → report
- Per-city writes overwrite only that city's partition (`replaceWhere`), so runs are idempotent and cities run in parallel. **Adding a city = one parameter value.**
- All logic lives in one Python package (`src/airbnb_lakehouse`); the notebooks parse widgets and call it, and the same code runs locally for tests
- Model versions are immutable files in a UC volume (+ MLflow), so "score Amsterdam with v1" really means the week-1 model

The model beats the baseline by 30 % in Brussels and Antwerp



- **Target**: log(price), converted back with exp. **Split**: deterministic 20 % test per listing (xxhash of id), held out before any choice
- **Choice**: 5-fold CV on the training split. Ridge €42.9, random forest €38.6, **gradient boosting €37.9**, baseline €51.7
- **Features** (all known before setting a price): guests, bedrooms, bathrooms, room/property type, minimum nights and long-stay flag, amenities, reviews, host, distance to the city centre, target-encoded neighbourhood, city
- Most important: guests, bedrooms, minimum nights, city, room type
- Weak spot: the expensive tail. Above €400 the MAE is €173 and prices are under-predicted

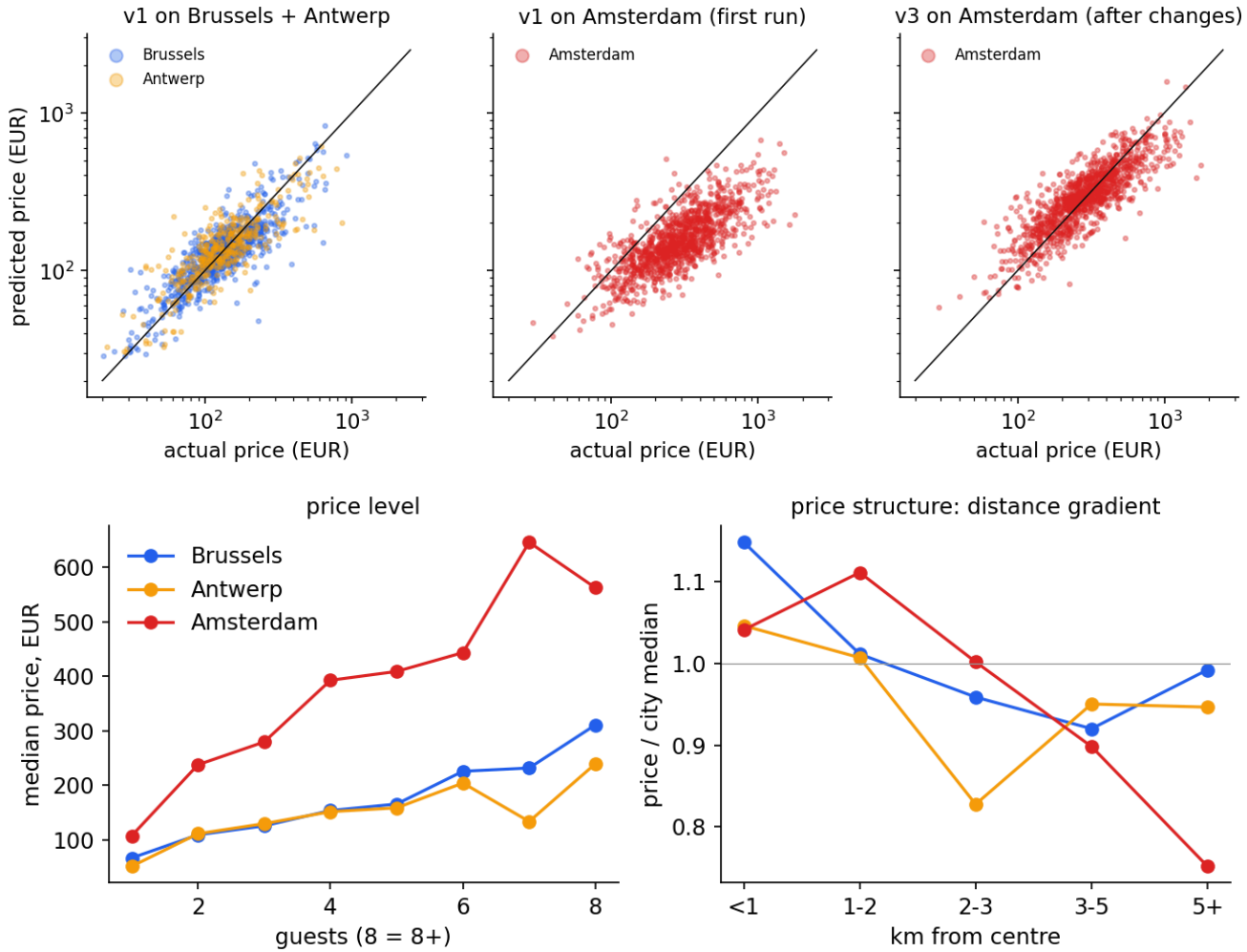
City C, first run: no errors, no warnings, and useless prices

- The frozen pipeline found Amsterdam on the index, downloaded it, typed and cleaned it: **exit code 0**
- Same 90 columns, EUR prices: every format check passed
- **MAE €158.60** on 5,927 priced listings (A+B: €34.91; A+B baseline on C: €195.54)
- 95 % of listings under-priced, mean error **−€155**; highest prediction €601
- Silently passed through: **43 %** missing prices (A/B: 16-20 %), **42 %** closed calendars, **100 %** unseen neighbourhoods, 2.2x price level, and 112 genuine homes flagged as "outliers"

	Brussels	Antwerp	Amsterdam
median price	€125	€130	**€285**
% calendars closed	13 %	12 %	**42 %**
% multi-listing hosts	53 %	61 %	**19 %**
% long-stay	8 %	7 %	**1 %**
price per guest	€48	€45	**€108**

The pipeline was robust to *format* changes and blind to *distribution* changes.

Why: a price level the trees never saw, and a different price structure



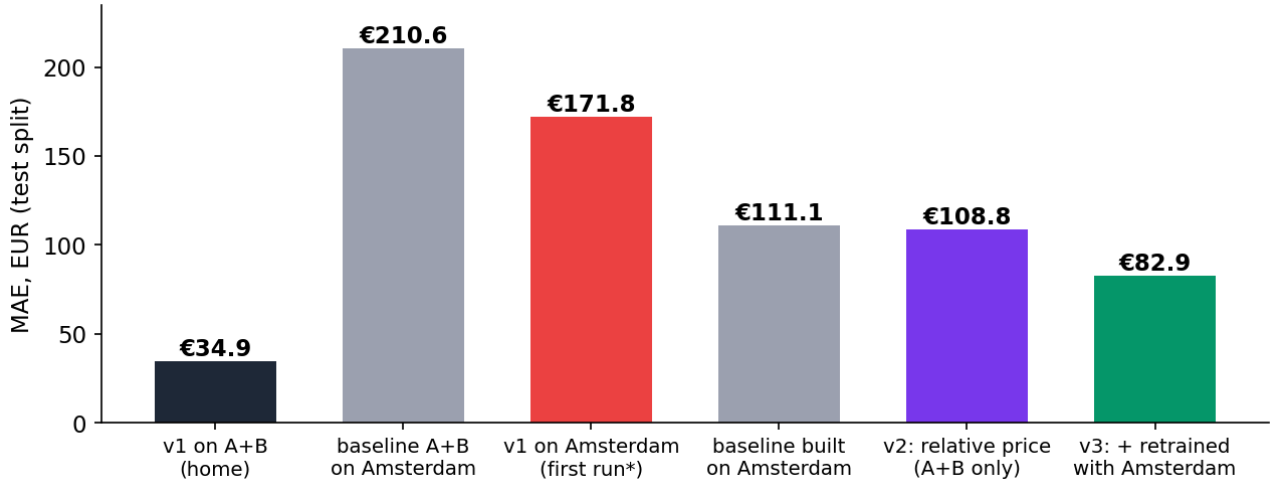
Trees cannot extrapolate: v1 predictions flatten around Belgian levels. A single oracle rescaling factor would halve v1's error. Amsterdam also differs in structure: per-guest prices are higher, and price falls faster away from the centre.

What we changed, and which kind of change it was

#	kind	change	why
P1	pipeline	outlier ceiling = 8x the city's median (was a fixed €1,000)	the rule encoded Belgian prices; it removed 103 genuine Amsterdam homes
P2	pipeline	data-quality expectations to dq_metrics (pass/warn)	43 % missing target went through silently; it now warns
P3	pipeline	scoring coverage report: unseen city, unseen neighbourhoods, price level	the model must say when it extrapolates
P4/P5	pipeline	dq_metrics per layer; city lists as `a+b+c` via a plan task	two writers to one table; the CLI splits params on commas
M1	model	relative target: log(price / city price index), with the index taken from the city's training split	learn structure, not Belgian euros
M2	model	drop the one-hot `city`	meaningless for an unseen city
M3	model	retrain with Amsterdam's 80 % training split (v3)	its structure differs; its 20 % test split stays untouched

Unchanged because they held up: URL resolution, download, column contract, try_cast typing, parsing, dedup, split.

The numbers: Amsterdam's MAE falls from €172 to €83, and the rest of the gap is proportional



- * €171.8 = v1 on Amsterdam's test split once the outlier fix puts the 103 expensive homes back. The original first run scored **€158.6** (all priced) / €156.0 (test split)
- **Price level ≈ €60 of the gap**: v1 x (Amsterdam median / A+B median) gives €112.7; v2 gives €108.8, no better than a trivial baseline built on Amsterdam's own data (€111.1)
- **Structure ≈ €26**: v3 learns Amsterdam's room-type premiums and distance gradient, 25 % better than that baseline
- **The rest is proportional**: MAPE is 26 % for v3 on Amsterdam and 23 % for v1 at home, so the same relative error lands on 2.2x higher prices
- Cost at home: v3 is €35.1 on Brussels (v1: €32.9), so in production we route A/B to v1 and C to v3

Dashboard and Genie (live demo)

AI/BI dashboard: 3 pages, each titled with a finding

- **City overview**: "Amsterdam costs 2.3x Belgium, and 42 % of its calendars are closed". Listings, price, availability, ratings, room mix, neighbourhoods by price
- **Cross-city comparison**: "Brussels and Antwerp price alike; what differs is the mix". Side-by-side metrics, price distributions, room-type prices, revenue
- **Model**: MAE table for every version and city, predicted vs actual, error by room type and neighbourhood, top under/over-priced listings, pipeline warnings

Genie: the tricks we used to configure it

- Only 5 pre-aggregated gold tables; medians precomputed with the dashboard's outlier rule, so answers match the dashboard
- Instructions map "City A/B/C" to slugs, "entire home" to `Entire home/apt`, and fix the sign of `error` (negative = under-priced)
- The honest-evaluation filter is written out (test split or new city) so Genie never reports training error
- Production routing (v1 for A/B, v3 for C) as the default model version
- UC comments on every column; trusted example SQL for each sample question

Challenges, and who did what

- **Holdout discipline**: freezing a git tag and a model hash before touching Amsterdam, so the first run is provably unchanged
- **Leaks hidden in new columns** (`price_quote_*`, revenue): found only by checking exact equalities, not correlations
- **Missing target**: 16-43 % of listings have no price, for structural reasons; deciding not to impute it
- **Free Edition constraints**: serverless only, ANSI SQL (hence try_cast everywhere), no cluster libraries (%pip per notebook)
- **Silent success** is harder to catch than a crash: a stack trace would have been easier to deal with than a confident wrong table

member	contribution
name	*fill in*
name	*fill in*
name	*fill in*